# LiveChat AI Bot: System Document

How the system works, what it is built with, how the database is designed, and how the screens are laid out.

*Written 5 November 2026. This describes the code in the repository* `yiling0923/aibot`.

---

## Contents


1. What the system is
2. The big picture
3. Technology used
4. How a customer message is handled
5. The bot engine (the rules)
6. The AI layer (optional)
7. The control panel
8. Login and security
9. Database design
10. Screens and layout
11. API addresses
12. Folders and files
13. Settings (.env)
14. How to run it
15. Testing and quality
16. What is not finished
17. What you asked for, and where it is
18. Installing on Windows (XAMPP) and fixing common problems


---

## 1. What the system is

This is a chatbot for a **LiveChat** account that serves **several merchants** (for example LuckyPay, BetKing). Each merchant has its own knowledge base, keywords, tone and settings.

What the bot does:

- Understands customers writing in **Bahasa Melayu, Chinese and English**, including slang and short forms (`dep`, `wd`, `tarik duit`, `充钱`).
- Finds the matching answer in **that merchant's own knowledge base** and replies in the customer's language and the merchant's tone (friendly or formal).
- When a message is not clear, **asks again once** with buttons. If it still does not understand, it **hands the chat to a human**.
- When a customer is rude, it **replies kindly first** and closes the chat only if the customer keeps being rude (default after 3 rude messages, set per merchant).

- Can look up a logged-in customer's **own** balance, transactions or points, but only through a signed login token. It **never shows** passwords, OTP codes or full bank, card or ID numbers.
- Never moves money. For amounts (for example "wd 500") it only checks the merchant's limits and explains the steps.

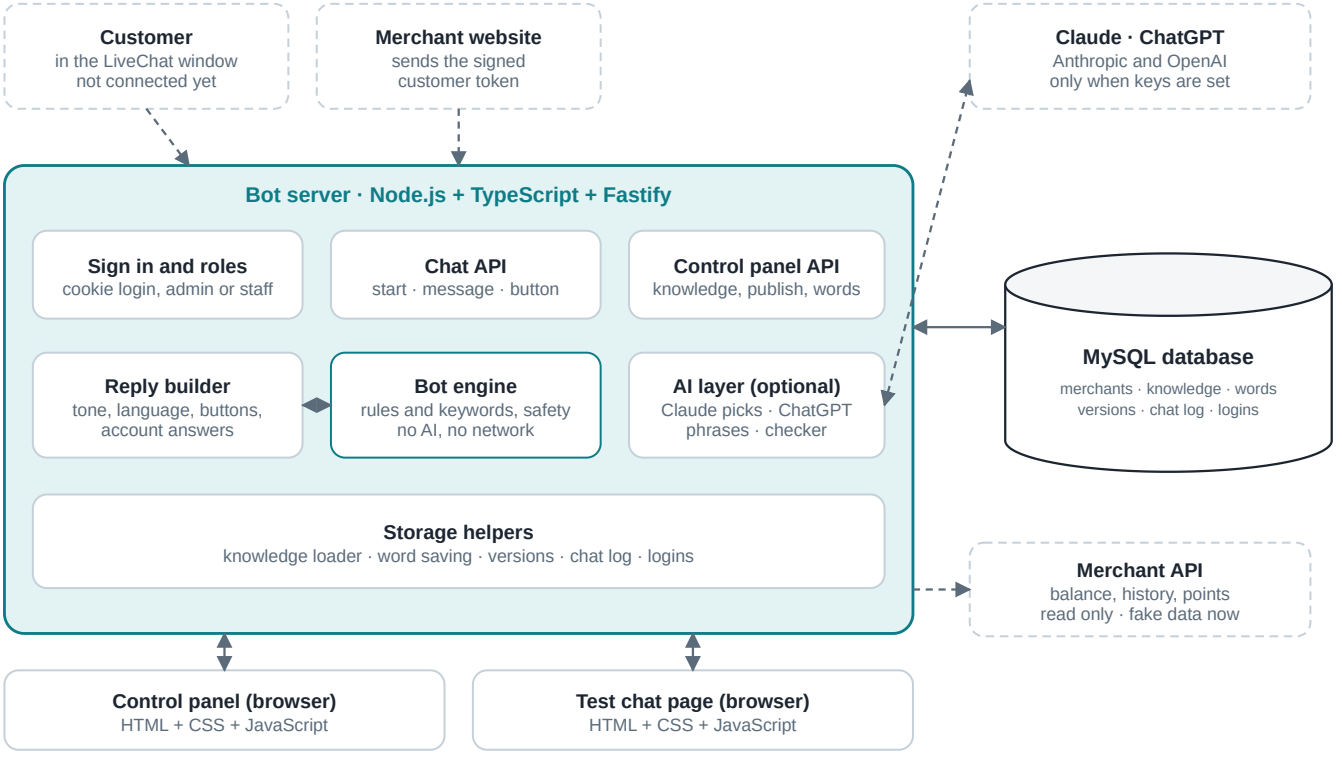
The system has two parts:

| Part | Who uses it | What it is |
|---|---|---|
| **The bot** | Customers (through LiveChat) | Receives a message, decides, replies |
| **The control panel** | Merchant staff and admins | A web page to manage knowledge, keywords, settings, versions and logins |

### Status in one view

| Done | Not finished yet |
|---|---|
| Database, bot engine, test chat page, control panel, logins, merchants and users screen, AI layer (built, tested with a fake network) | Connection to the real LiveChat, testing the AI with real keys, hosting on a real server |

## 2. The big picture




| Block | What it does |
|---|---|
| **Bot server** | One Node.js program. It serves the pages, answers the chat, and runs the control panel's back end. |

| Block | What it does |
|---|---|
| **Sign in and roles** | Checks the login cookie on every request. An admin sees everything; staff only their own merchant. |
| **Chat API** | Starts a chat, receives a typed message, receives a tapped button. |
| **Control panel API** | Reads and saves the knowledge base, publishes versions, lists keywords, manages merchants and logins. |
| **Reply builder** | Turns the engine's decision into the messages and buttons the customer sees (language, tone, account answers). |
| **Bot engine** | The decision logic. Pure rules, no AI and no network. |
| **AI layer (optional)** | Claude can pick an entry when no keyword matches, ChatGPT can phrase a reply, and a checker guards the facts. Off until keys are added. |
| **Storage helpers** | Load the knowledge from the database into memory, save the words customers type, write versions and logins. |
| **MySQL database** | Keeps everything: merchants, knowledge, words, versions, logins. |
| **Merchant API** | Where each merchant's account data lives (balance, transactions, points). Read only. Fake JSON data for now. |
| **Browser pages** | The control panel, the test chat page and the login page. |

Dashed boxes in the picture are not connected yet or are optional.

## 3. Technology used

| Layer | Technology | Notes |
|---|---|---|
| **Programming language** | **TypeScript 7** on **Node.js 22** (works on Node 20 or newer), ES modules | One language for the whole back end. TypeScript catches mistakes before the program runs. |
| **Web server** | **Fastify 5** with `@fastify/static` | Fastify answers the API calls; the static plugin serves the built front end (the `web/dist` folder). |
| **Database** | **MySQL 8** | Also works on MariaDB 10.11 (the version in XAMPP). Tables use InnoDB and `utf8mb4`, so Chinese text is stored correctly. |
| **Database driver** | `mysql2` (promise pool) | Every query uses placeholders ( `?` ), so customer text can never change a query. |
| **Settings** | `dotenv` | Reads the `.env` file. |
| **AI** | `@anthropic-ai/sdk` (Claude) and `openai` (ChatGPT), official libraries | Answers from Claude are checked with **zod** (a schema checker) before use. |
| **Passwords and signatures** | Node's built-in `crypto` | scrypt for password hashing, HMAC-SHA256 for signed cookies and tokens. |

| Layer | Technology | Notes |
|---|---|---|
| **Tests** | **Vitest 5**, run with `tsx` (front end tests use **jsdom** and **Testing Library**) | 2,017 automatic tests. |
| **Front end** | **React 19 + TypeScript + Vite 8**, with **React Router 7** | A real single-page app. Vite builds it into `web/dist`, and the bot server serves that folder. The front end shares its data types with the back end (`src/admin/snapshotTypes.ts`), so a mismatch is caught when the code is checked. |
| **Front end look** | CSS variables, Google Fonts (Instrument Sans, JetBrains Mono), inline SVG icons | Light and dark mode follow the computer's setting. Works on phones. |

### Why these choices

- **One language (TypeScript)** keeps the project simple to maintain.
- **React + TypeScript + Vite** gives the screens proper components, typed data and a normal build, so the control panel can keep growing without turning into one big script. Each page (login, chat, control panel) is loaded on its own.
- **MySQL** was chosen because the existing tools (XAMPP) already include it.
- **The engine has no AI and no network**, so it is fast, free to run, predictable, and can be tested with thousands of sample messages.

### Size of the code

| Area | Lines |
|---|---|
| Bot engine | about 380 |
| Reply builder and texts | about 490 |
| AI layer | about 190 |
| Control panel back end | about 510 |
| Login | about 120 |
| Database helpers and server | about 430 |
| Control panel front end (React, `web/src/admin`) | about 1,500 |
| Test chat page, login page, shared code and style sheets (React, `web/src`) | about 900 |
| Automatic tests | about 860 |

# 4. How a customer message is handled




The steps are checked **in this order**. The first one that fits decides the reply.

### The ten decisions the bot can make

| Decision | Meaning | What the customer sees |
|---|---|---|
| **ANSWER** | One knowledge base entry matches | The answer, in the customer's language and the merchant's tone |

| Decision | Meaning | What the customer sees |
|---|---|---|
| **CHOICE** | Several entries could match, or only a topic was named (for example "dep") | Buttons: "How to deposit", "Deposit time", "Deposit limits", "Talk to an agent" |
| **CLARIFY** | Only a question word ("how long?") or two close matches | Two buttons to pick from |
| **ASK AGAIN** | Nothing matched (first time) | "Sorry, I didn't quite catch that" with topic buttons |
| **HANDOFF** | The bot cannot or should not answer | A message, then the chat goes to the merchant's human group |
| **ABUSE** | Rude words | A kind reply; after N rude messages the chat is closed |
| **SMALLTALK** | Greeting, thanks, goodbye, or only filler words | A friendly reply (never a handoff) |
| **ACCOUNT** | Asking for own balance, transactions or points | The customer's own data, only with a valid login token |
| **REFUSE** | Asking for a password, OTP or full card number | A polite refusal and an agent button |
| **AMOUNT** | A number such as "cash 500" | Asks what to do, or checks the limits |

(ASK AGAIN is added by the reply builder; the other nine come from the engine.)

**What the bot remembers during one chat** (kept in the server's memory):

- how many times it asked for clarification and how many times it did not understand,
- how many rude messages there were,
- the last topic (so "500" after a deposit question can be understood),
- an earlier unclear message, which is read together with the next one,
- whether the chat was handed to a human or closed.

After a handoff the bot stays quiet. After a close it accepts no more messages.

---

## 5. The bot engine (the rules)

The engine is a plain TypeScript class (`src/engine/engine.ts`). It receives the merchant's knowledge once, keeps it in memory, and answers each message in a fraction of a millisecond. It does not call the internet.

**Step 1: normalise the message**

- lower-case the text and detect the language (English, Malay or Chinese),
- replace short forms with the real word (`dep` becomes `deposit`, `wd` becomes `withdraw`), using shared words and the merchant's own words,
- remove greetings at the start (`boss, depo how` is read as `depo how`) and filler words (`lah`, `loh`, `啦`, `please`).

**Step 2: find keywords anywhere in the sentence**

- **Topic words** say what the question is about (deposit, withdraw, password, promotion, turnover, refund...).
- **Question words** say what the customer wants to know (how, time, limit, change, opening hours).

- Both can be anywhere in the sentence, and the words around them are ignored. "I nak wd" works because `wd` is found.
- The **longest phrase wins** ("what time" is opening hours, "how much" is a limit, not "how").
- A number with a unit ("2 hours") counts as time when a topic is also named.
- A knowledge base entry needs a **topic word**. The **question type is optional** (ticked per entry).

**Step 3: safety checks (run early)**

- **Rude words**: strong words always count; mild words only when alone or with other insults; spelled-out forms like `p u k i m a` are caught too.
- **Rule-change attempts** ("ignore your rules"), **another merchant's name**, and **requests for secrets** (password, OTP, full card number).
- **Asking for a person** ("talk to agent", `转人工`) hands over straight away.
- **Problem words** (rejected, failed, complaint) go to a human, because the bot cannot see why.

**Step 4: pick the answer**

- One matching entry becomes an **ANSWER**; several, or only a topic, become **CHOICE** buttons.
- A topic the merchant has no entry for goes to a human instead of showing confusing buttons.
- If no keyword matches, the engine compares the sentence with every **example question** of the merchant (a similarity score from 0 to 1). A score of **0.72 or more** answers, **0.45 or more** asks with buttons. Both limits can be changed per merchant.

**Account questions**

- Balance, transactions and points are answered only when the merchant has switched that tool on **and** the website sent a valid signed customer token.
- The customer id comes only from the signed token, never from what the customer types, so nobody can ask about another customer.
- The data comes from the merchant API (fake JSON data in this version).

**Replies**

All fixed texts (greetings, buttons, refusal, limits, account answers) exist in the three languages and in two tones, **friendly** and **formal**. The answer text itself comes from the merchant's knowledge base; the tone only adds a short opening and closing.

---

# 6. The AI layer (optional)

**Important: the bot never invents an answer.** Every answer a customer sees comes from an entry that a person wrote and published in the knowledge base. The AI does not search for or write new facts:

- **Claude only chooses.** When the message is not clear, it picks the closest entry from the entries that already exist, or says it is not sure. It writes nothing.
- **ChatGPT only rewords.** It takes the written answer and says it in the merchant's tone (friendly or formal). Numbers, steps and amounts must stay the same. A checker compares every reworded reply with the original, and if a fact changed the original wording is used instead.
- **No entry, no answer.** If nothing matches, the bot asks once more and then hands the chat to a person.

- **So each merchant needs its answers set up first** (deposit, withdraw, limits, forgot password, promotions...). It is not necessary to list every way a customer can write a question: a topic word plus at least 3 example questions is enough, and short forms such as `depo` or `dep` are recognised.
- The "Suggest with AI" button in the entry editor is a **placeholder** for now: it only adds simple variations of the first example. It does not call an AI.
- Writing draft answers automatically from a merchant's documents is **not built** and not planned for now.

The AI is **off until keys are added** to the `.env` file. With no keys the bot works exactly as described above. Each AI step also falls back safely if the service is slow, down or returns something unusable.

| Step | Service | What it does | Safety |
|---|---|---|---|
| **Find the entry** | Claude (default model `claude-haiku-5-5`) | Only when no keyword matched: asks which knowledge base entry the message is about | The answer must be a real entry and at least **75% sure**. Otherwise ignored. |
| **Phrase the reply** | ChatGPT (model set in `.env`) | Rewrites the knowledge base answer as a natural reply in the customer's language and tone | A **checker** compares the rewrite with the original: every number must stay, and no new number, link or menu path may appear. If it fails, the standard wording is used. |
| **Describe new keywords** | Claude | Writes a short note and suggests a type for each new word (filler, short form, topic, question, rude, greeting...) | Staff make the final decision. Notes written by hand are never replaced. |

### Protection

- The customer's text is always passed as *data*, with an instruction not to follow anything inside it.
- **Account data is never sent** to the AI. Only the customer's message and the knowledge base text.
- A **timeout** (8 seconds, one retry) and a limit of **600 AI calls per hour** (changeable) protect cost and speed.
- Claude's answers are checked with a schema (zod), and a made-up entry name is ignored.

Status: built and tested with a **fake network** (21 tests). It has not been tried with real keys yet.

---

## 7. The control panel

The control panel is a web page at `/admin`. Staff change the bot here without touching code.

### Screens

| Group | Screen | What it is for |
|---|---|---|
| **Answers** | Knowledge base | The entries the bot may answer: name, topic, question type, examples and answer in three languages, live or draft |
| | Import | Load many entries from a CSV file (sample file and template included), with a check before importing |
| **Words** | Topics | Topic keywords and short forms, question words, greetings, fillers and block lists (rude words, safety lists, asking for a person) |

| Group | Screen | What it is for |
|---|---|---|
| | Keywords | Every word customers typed, saved automatically, newest first. Make a word a topic, add it to a topic, or ignore it. A short AI note appears when AI is on. |
| | Keyword ranking | The words customers wrote most in a week: top 3 as medal cards, then a table with trend and change |
| **Check** | Test chat | Chat with the bot using your **unpublished** changes. Shows every step the bot took. |
| | Tests | Runs the sample customer messages and shows what passed or failed |
| **Setup** | Merchant settings | Tone, handoff group, answer limits, account tools, how many rude messages before closing, fixed messages |
| | Merchants and logins (admin) | Add a merchant (starts empty), add or remove logins, reset passwords |
| | History | Every published version, with roll back |

### Draft, test, publish

1. Edits stay a **draft in the browser** (saved there, so a refresh does not lose them).
2. **Test chat** uses a temporary copy of the bot built from the draft. The live bot is not touched.
3. **Publish** runs all sample messages on the draft first and shows the result. Then it saves everything in **one database transaction**, writes a **new version** (a full copy of the knowledge) and reloads the live bot in memory.
4. **Roll back** restores an older version of **one merchant**. Other merchants are not touched. Staff can roll back their own merchant; the shared word lists can only be rolled back by an admin.

### Roles

| Role | Sees | Can change |
|---|---|---|
| **Admin** | All merchants, all screens | Everything, plus merchants, logins and the shared word lists |
| **Staff** | Only their own merchant (the merchant is fixed, there is no picker) | Their own entries, short forms, tone and fixed messages |

The server enforces this; hiding a button is not what protects the data (see section 8).

---

## 8. Login and security

| Topic | How it works |
|---|---|
| **Login** | Username and password. Passwords are stored only as **scrypt hashes**, never in plain text. |
| **Session** | A signed cookie (HMAC-SHA256), `HttpOnly`, valid for 12 hours. It cannot be changed without the secret. |
| **Wrong passwords** | After 5 wrong tries for the same user and address, login is blocked for a minute. A wrong username takes as long as a wrong password. |
| **Every page and call** | Needs a valid session. Without one, pages go to the login page and API calls return 401. Only login and the health check are open. |

| Topic | How it works |
|---|---|
| **Merchant lock** | For staff, the server (a) returns only their merchant's data, (b) ignores everything else in a publish request (other merchants, shared word lists, limits, tools), and (c) refuses other merchants' keywords, ranking and chat. This is tested by sending forged requests. |
| **Customer token** | The merchant website signs `customer_id` and name with a shared secret (HMAC-SHA256). The bot trusts the id only if the signature matches. A forged or edited token is ignored. |
| **Secrets** | The bot never shows passwords, OTP codes or full bank, card and ID numbers, even to a verified customer. |
| **Database** | All queries use placeholders. |
| **AI** | See section 6: untrusted-text rule, no account data sent, fact checker, call limit. |
| **Roll back** | Staff: their own merchant. Admin: any merchant and the shared word lists. Reload is admin only. |
| **Version history** | Records who published each version. |

Before real use: change the demo passwords, set `SESSION_SECRET` and `CUSTOMER_TOKEN_SECRET` to long random values, turn `DEV_TOOLS` off, and put the server behind HTTPS (see section 16).

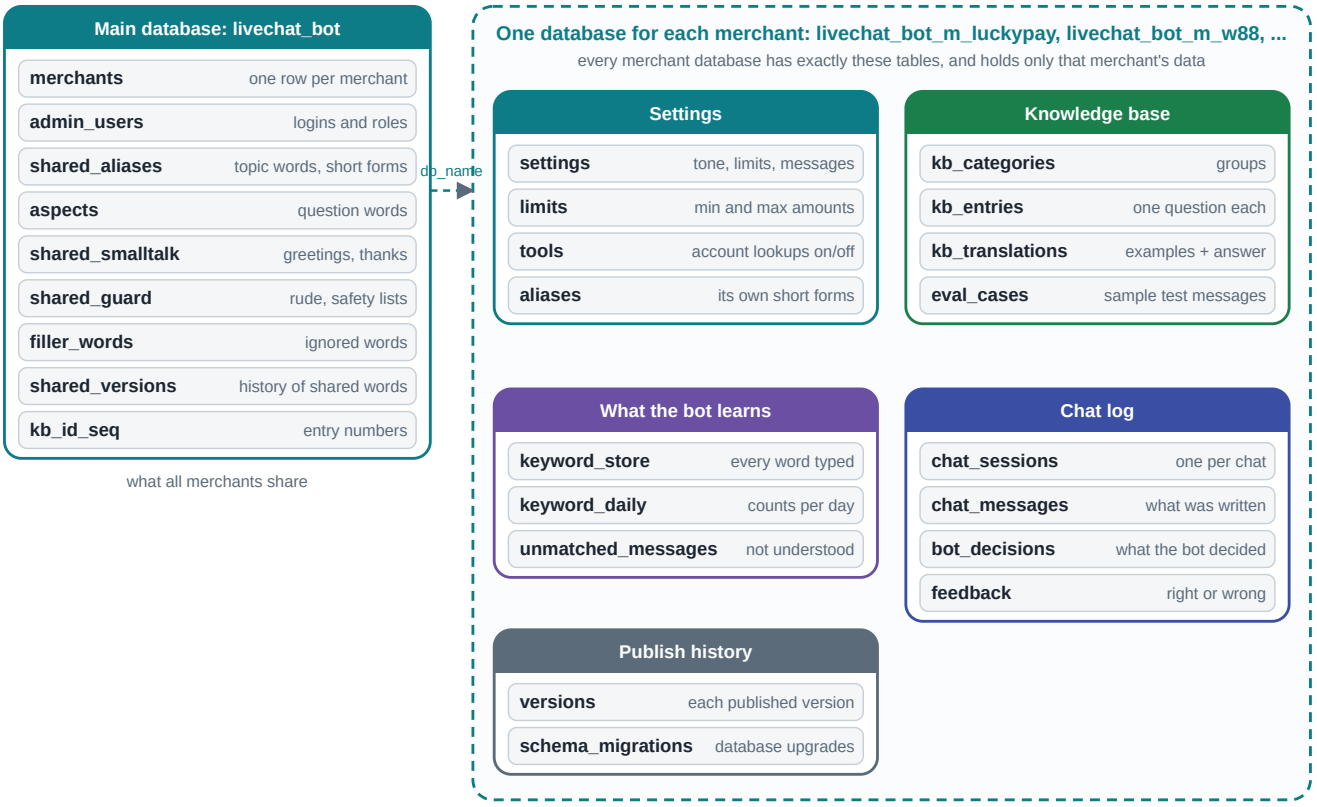
---

# 9. Database design

### Principles

- **A main database, and one database for each merchant.** The main database (`livechat_bot`) holds only what is shared: the merchant list (`merchants`), the logins, the word lists shared by all merchants, and their version history. Every merchant has its **own database**, named `livechat_bot_m_<code>` (for example `livechat_bot_m_luckypay`). It holds that merchant's knowledge base, settings, own short forms, keywords, unmatched messages, chats, sample tests and publish history.
- **Inside a merchant database there is no `merchant_id`**: the database itself is the merchant, so one merchant's data can never be mixed with another's, and a query can only ever reach the database it names.
- **Adding a merchant creates its database** with all tables (from the files in `db/merchant`). The list of merchants records the database name in `merchants.db_name`.
- **Languages** are stored per row (`en`, `ms`, `zh`), not as separate columns, so a language can be added without changing the tables.
- **Flexible parts use JSON columns** (example questions, fixed messages, settings), so they can grow without new columns.
- **Versions are kept per merchant.** Publishing writes a new row in that merchant's own `versions` table only if its data really changed. The shared word lists have their own history (`shared_versions`). Each merchant counts its own `v1`, `v2`, ... and is rolled back on its own; other merchants are not touched.
- **Entry ids never repeat across merchants.** New entries get their number from `kb_id_seq` in the main database, so the control panel can find an entry by its id.
- **Numbered migrations** create and change the tables: `001` to `008` in `db/migrations` for the main database (`008` is a one-time step that moved the data of an older single-database installation into merchant databases), and the files in `db/merchant` for every merchant database. Each database has a `schema_migrations` table that records which ones have run, so running them again is safe.

- **The database user needs one grant**, in `db/setup.sql`: the right to create and use databases that start with `livechat_bot_`. It is run once as the MySQL admin.
- Text is stored as **utf8mb4** so Chinese, Malay and emoji work.




### How the main parts work together

- The **main database** knows who the merchants are and where each one's database is, who may sign in (and for which merchant), and the words every merchant shares (`shared_aliases`, `aspects`, `shared_smalltalk`, `shared_guard`, `filler_words`).
- In a **merchant database**: the merchant's settings (`settings`), amount limits (`limits`) and account tools (`tools`); the **knowledge base** (`kb_entries` with a topic and an optional question type, grouped by `kb_categories`, with the examples and the answer for each language in `kb_translations`); its own short forms (`aliases`); **what the bot learns** (every typed word goes to `keyword_store` and `keyword_daily`, which the ranking uses); the **chat log** (`chat_sessions`, `chat_messages`, `bot_decisions`); the sample customer messages with the result the bot should give (`eval_cases`, the automatic tests); and the publish history (`versions`).

**Seeing it in phpMyAdmin.** phpMyAdmin can group databases by their name. Add this line to `config.inc.php` (below the `host` line) and the left list shows `livechat_bot` with all the merchant databases under it:

```
$cfg['NavigationTreeDbSeparator'] = '_m_';
```

Without it the databases are simply listed one after another (`livechat_bot`, `livechat_bot_m_luckypay`, ...).

## The main database (`livechat_bot`)

There are **10 tables**. For each column: the type, and what is special about it (primary key, required, default, link to another table, unique).

### Merchants and logins

**`merchants`**. One row per merchant: its code (for example `MERCH_D`), name, main language, the LiveChat group and the human group chats are handed to, and `db_name`, the name of the merchant's own database.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `code` | varchar(50) | unique; required |
| `name` | varchar(200) | required |
| `status` | varchar(20) | default `active` |
| `default_language` | varchar(10) | default `en` |
| `livechat_group_id` | int | |
| `handoff_group_id` | int | |
| `created_at` | timestamp | default `CURRENT_TIMESTAMP` |
| `db_name` | varchar(64) | unique |

Indexes: `code` (unique), `db_name` (unique).

**`admin_users`**. Logins: username, password hash (scrypt), role (`admin` or `staff`), and for staff the merchant they belong to.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `username` | varchar(50) | unique; required |
| `password_hash` | varchar(300) | required |
| `role` | varchar(10) | required |
| `merchant_id` | bigint | links to `merchants.id` |
| `created_at` | timestamp | default `CURRENT_TIMESTAMP` |

Indexes: `merchant_id`, `username` (unique).

### Words shared by all merchants

**`shared_aliases`**. Topic keywords and short forms. `canonical` is the topic name (for example `deposit`) and `variant` is a way to write it (`dep`, `depo`, `充值`). A merchant's own short forms are in its own database.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `language` | varchar(10) | |
| `variant` | varchar(200) | required |
| `canonical` | varchar(100) | required |
| `weight` | double | default `1` |
| `source` | varchar(20) | default `manual` |
| `approved` | tinyint(1) | default `1` |

**`aspects`**. Question words by kind: `time`, `limit`, `how`, `change`, `hours`, plus `problem` (goes to a human) and `help` (ignored).

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `kind` | varchar(30) | required |
| `language` | varchar(10) | |
| `keyword` | varchar(200) | required |

**`shared_smalltalk`**. Greetings, thanks, goodbyes, calling for attention. `kind` says which.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `kind` | varchar(30) | required |
| `language` | varchar(10) | |
| `phrase` | varchar(255) | required |

**`shared_guard`**. The safety lists: rude words, mild words, rule-change attempts, secret requests, and asking for a person.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `kind` | varchar(30) | required |
| `language` | varchar(10) | |
| `phrase` | varchar(255) | required |

**`filler_words`**. Words with no meaning of their own (`lah`, `啦`, `please`). They are ignored when looking for keywords.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `language` | varchar(10) | |
| `word` | varchar(100) | required |

### Housekeeping

**`shared_versions`**. Every published version of the shared word lists: number, who published, a short summary, and the lists as one JSON document so they can be rolled back to.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `version` | int | unique; required |
| `actor` | varchar(100) | required |
| `summary` | varchar(500) | required |
| `snapshot` | longtext | required |
| `created_at` | timestamp | default `CURRENT_TIMESTAMP` |

Indexes: `version` (unique).

**`kb_id_seq`**. Hands out entry numbers, so an entry id never repeats across merchants (the control panel finds entries by id).

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |

**`schema_migrations`**. Which migration files have already run.

| Column | Type | Notes |
|---|---|---|
| `name` | varchar(200) | primary key |
| `applied_at` | timestamp | default `CURRENT_TIMESTAMP` |

## Inside each merchant database (for example `livechat_bot_m_luckypay`)

Every merchant database has these tables and holds only that merchant's data. The tables are described once here; the tables shown are from the sample merchant `MERCH_A`.

There are **17 tables**. For each column: the type, and what is special about it (primary key, required, default, link to another table, unique).

### Settings

**`settings`**. How the bot behaves for this merchant: tone, the two similarity limits, how many rude messages before closing, and the fixed messages (as JSON, one text per language). Always one row.

| Column | Type | Notes |
|---|---|---|
| `id` | tinyint | primary key |
| `tone` | varchar(20) | default `friendly` |
| `answer_threshold` | double | default `0.72` |
| `clarify_threshold` | double | default `0.45` |
| `abuse_close_after` | int | default `3` |
| `closing_msg` | json | required |
| `handoff_msg` | json | required |
| `kind_reply` | json | required |
| `business_hours` | json | |

**`limits`**. Minimum and maximum amount for deposit and withdraw. Used to answer "wd 500".

| Column | Type | Notes |
|---|---|---|
| `kind` | varchar(20) | primary key |
| `min_amount` | decimal(18,2) | required |
| `max_amount` | decimal(18,2) | required |

**`tools`**. Which account lookups are switched on (balance, transactions, points, deposit and withdraw status).

| Column | Type | Notes |
|---|---|---|
| `tool` | varchar(50) | primary key |
| `enabled` | tinyint(1) | default `1` |

**`aliases`**. This merchant's own short forms for topic words ( `canonical` is the topic, `variant` the way to write it).

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `language` | varchar(10) | |
| `variant` | varchar(200) | required |
| `canonical` | varchar(100) | required |
| `weight` | double | default `1` |
| `source` | varchar(20) | default `manual` |
| `approved` | tinyint(1) | default `1` |

## Knowledge base

**`kb_categories`**. Groups of entries (Deposit, Withdraw, Support...). Used for display.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `parent_id` | bigint | links to `kb_categories.id` |
| `name` | varchar(200) | unique; required |

Indexes: `name` (unique), `parent_id`.

**`kb_entries`**. One answerable question. `topic` is required, `aspect` is the optional question type, `status` is `live` or `draft`. Two entries cannot have the same `intent_key`.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `category_id` | bigint | links to `kb_categories.id` |
| `intent_key` | varchar(100) | unique; required |
| `title` | varchar(255) | |
| `tags` | json | |
| `topic` | varchar(100) | required |
| `aspect` | varchar(50) | |
| `topic_optional` | tinyint(1) | default `0` |
| `status` | varchar(20) | default `live` |
| `version` | int | default `1` |
| `valid_from` | timestamp | |
| `valid_to` | timestamp | |
| `updated_at` | timestamp | default `CURRENT_TIMESTAMP` |

Indexes: `category_id`, `status`, `intent_key` (unique).

**`kb_translations`**. For each entry and language: the example questions (JSON list) and the answer text.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `kb_id` | bigint | links to `kb_entries.id`; required |
| `language` | varchar(10) | required |
| `question_examples` | json | required |
| `answer_text` | text | required |

Indexes: `kb_id, language` (unique).

**`eval_cases`**. Sample customer messages and the result the bot should give (`expected_decision`, and for answers `expected_intent`). These are the automatic tests.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `message` | text | required |
| `language` | varchar(10) | |
| `expected_decision` | varchar(20) | required |
| `expected_kb_id` | bigint | links to `kb_entries.id` |
| `expected_intent` | varchar(100) | |

Indexes: `expected_kb_id`.

### What the bot learns

**`keyword_store`**. Every word customers typed: how many times (`hits`), when first and last seen, its status (`new` or `ignored`), where it came from (`live` or `test`), a short note and a suggested type.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `word` | varchar(100) | unique; required |
| `language` | varchar(10) | |
| `hits` | int | default `1` |
| `status` | varchar(20) | default `new` |
| `first_seen` | timestamp | default `CURRENT_TIMESTAMP` |
| `last_seen` | timestamp | default `CURRENT_TIMESTAMP` |
| `source` | varchar(20) | default `live` |
| `note` | varchar(200) | |
| `guess` | varchar(20) | |

Indexes: `word` (unique).

**`keyword_daily`**. How many times each word was written on each day. The Keyword ranking screen adds these up per week.

| Column | Type | Notes |
|---|---|---|
| `word` | varchar(100) | primary key (with day) |
| `day` | date | primary key (with word) |
| `hits` | int | default `0` |

**`unmatched_messages`**. Messages the bot did not understand, counted when repeated.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `text` | varchar(500) | unique; required |
| `language` | varchar(10) | |
| `hits` | int | default `1` |
| `resolved` | tinyint(1) | default `0` |
| `first_seen` | timestamp | default `CURRENT_TIMESTAMP` |
| `last_seen` | timestamp | default `CURRENT_TIMESTAMP` |
| `suggestion` | varchar(300) | |
| `resolution` | varchar(100) | |

Indexes: `text` (unique).

### Chat log

**`chat_sessions`**. One row per chat: language, status.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `livechat_chat_id` | varchar(100) | unique |
| `customer_ref` | varchar(200) | |
| `language` | varchar(10) | |
| `status` | varchar(20) | |
| `clarify_count` | int | default `0` |
| `closed_reason` | varchar(100) | |
| `started_at` | timestamp | default `CURRENT_TIMESTAMP` |

Indexes: `livechat_chat_id` (unique).

**`chat_messages`**. What was written in a chat.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `session_id` | bigint | links to `chat_sessions.id`; required |
| `livechat_event_id` | varchar(100) | unique |
| `role` | varchar(20) | |
| `text` | text | |
| `normalized_text` | text | |

| Column | Type | Notes |
|---|---|---|
| `created_at` | timestamp | default `CURRENT_TIMESTAMP` |

Indexes: `created_at`, `livechat_event_id` (unique), `session_id`.

**`bot_decisions`**. What the bot decided for a message (decision, matched entry, confidence, reason, reply text).

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `message_id` | bigint | links to `chat_messages.id` |
| `decision` | varchar(20) | required |
| `matched_kb_id` | bigint | links to `kb_entries.id` |
| `confidence` | double | |
| `candidates` | json | |
| `reply_text` | text | |
| `claude_ms` | int | |
| `gpt_ms` | int | |
| `validator_ok` | tinyint(1) | |
| `why` | varchar(500) | |
| `reviewed` | varchar(100) | |
| `created_at` | timestamp | default `CURRENT_TIMESTAMP` |

Indexes: `matched_kb_id`, `message_id`.

**`feedback`**. For marking a decision right or wrong. Not used by any screen yet.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `decision_id` | bigint | links to `bot_decisions.id` |
| `agent_id` | varchar(100) | |
| `verdict` | varchar(20) | |
| `correct_kb_id` | bigint | links to `kb_entries.id` |
| `note` | text | |

Indexes: `correct_kb_id`, `decision_id`.

## Publish history

**`versions`**. Every published version of this merchant: number, who published, a short summary, and the merchant's knowledge as one JSON document so it can be rolled back to.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | primary key; numbers itself |
| `version` | int | unique; required |
| `actor` | varchar(100) | required |
| `summary` | varchar(500) | required |
| `snapshot` | longtext | required |
| `created_at` | timestamp | default `CURRENT_TIMESTAMP` |

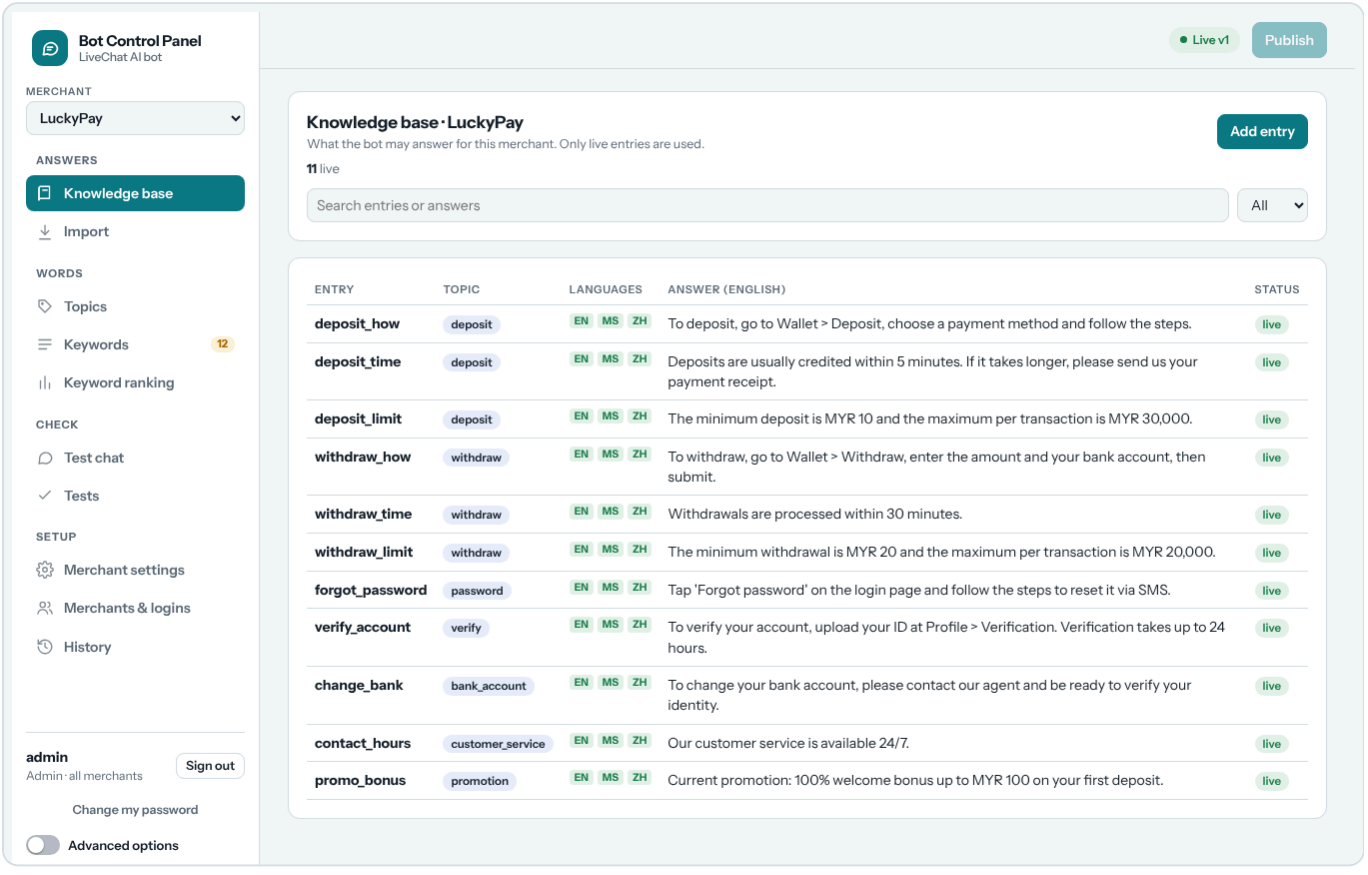
Indexes: `version` (unique).

**`schema_migrations`**. Which migration files have already run for this merchant database.

| Column | Type | Notes |
|---|---|---|
| `name` | varchar(200) | primary key |
| `applied_at` | timestamp | default `CURRENT_TIMESTAMP` |

# 10. Screens and layout

## Layout of the control panel

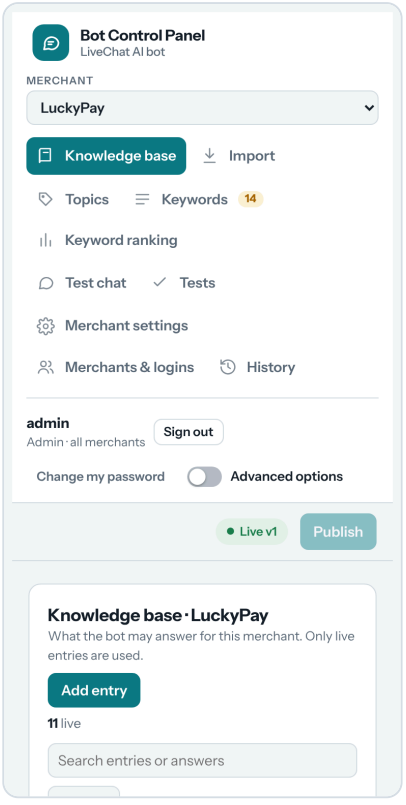


The panel has **three areas**:

1. **Left sidebar** (fixed, 248 px) - the logo and name, - the **merchant**: a picker for admins, a fixed name for staff, - the **menu in four groups**: *Answers*, *Words*, *Check* and *Setup*, each item with a small icon and a count badge where something needs attention, - at the bottom: who is signed in, **Sign out**, **Change my password**, and the **Advanced options** switch.
2. **Top bar**: the live version (`Live v1`), a draft chip with **Discard** when there are unpublished changes, and the **Publish** button.
3. **Content area**: uses the **full width**. Each screen is made of white cards on a light grey background, with one clear heading per card.

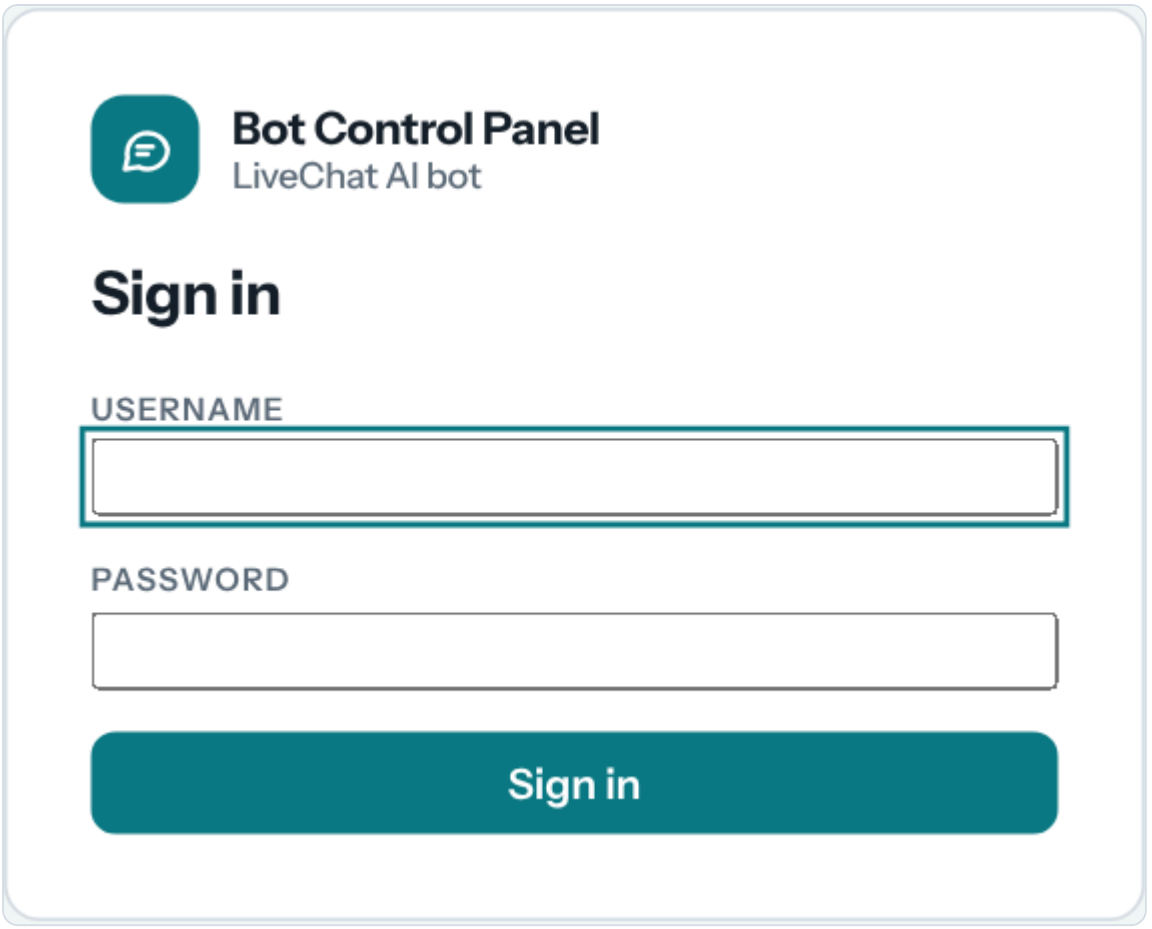
Other design rules:

- **One accent colour** (teal) for the main action; green, amber and red only for status.
- **Light and dark mode** follow the computer's setting.
- The page you are on is kept in the address bar (`#rank`), so a **refresh stays on the same page** and the Back button works.
- On a **phone**, the sidebar moves above the content and the menu wraps; the page never scrolls sideways.



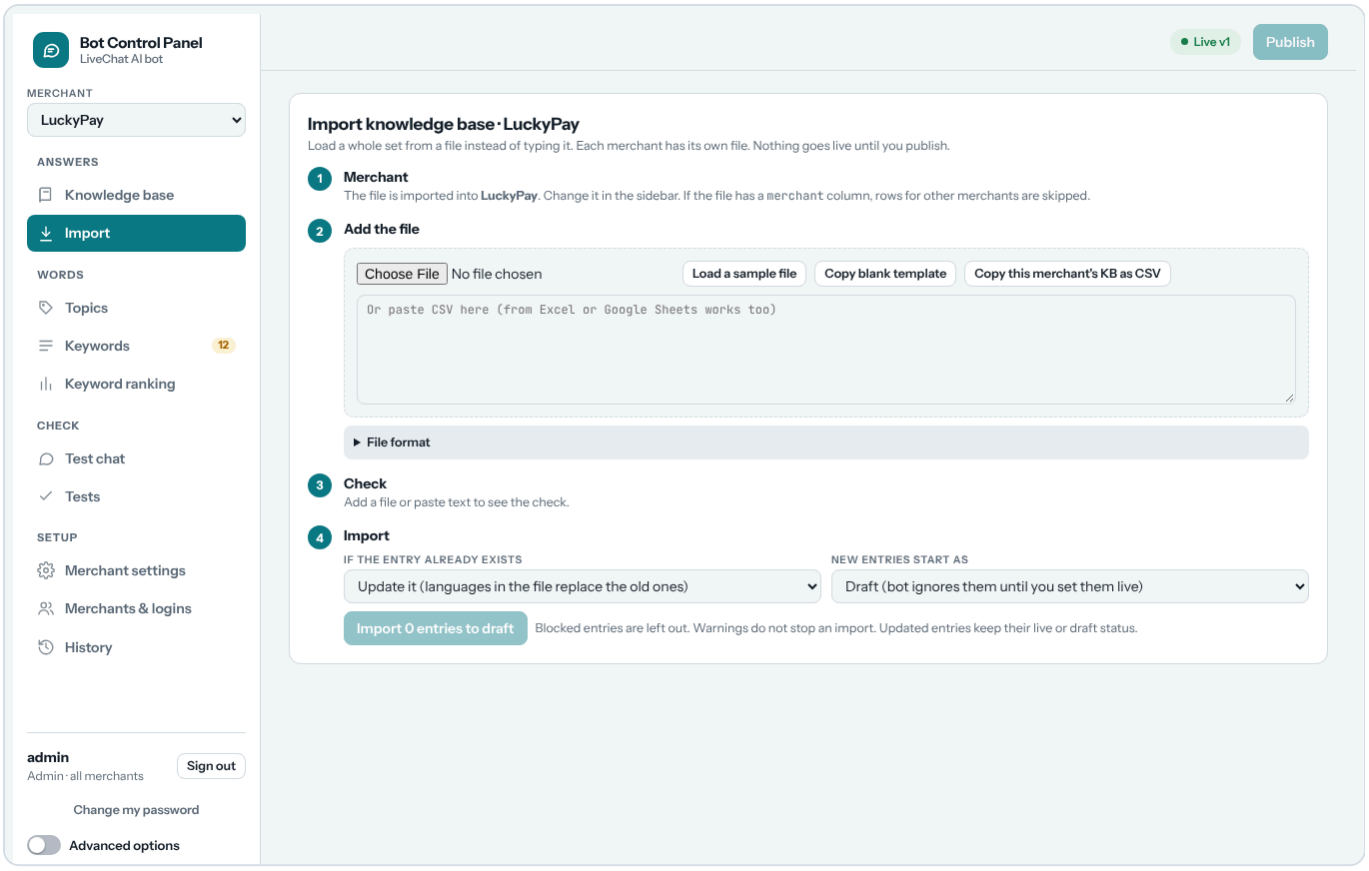

## The screens

### Login

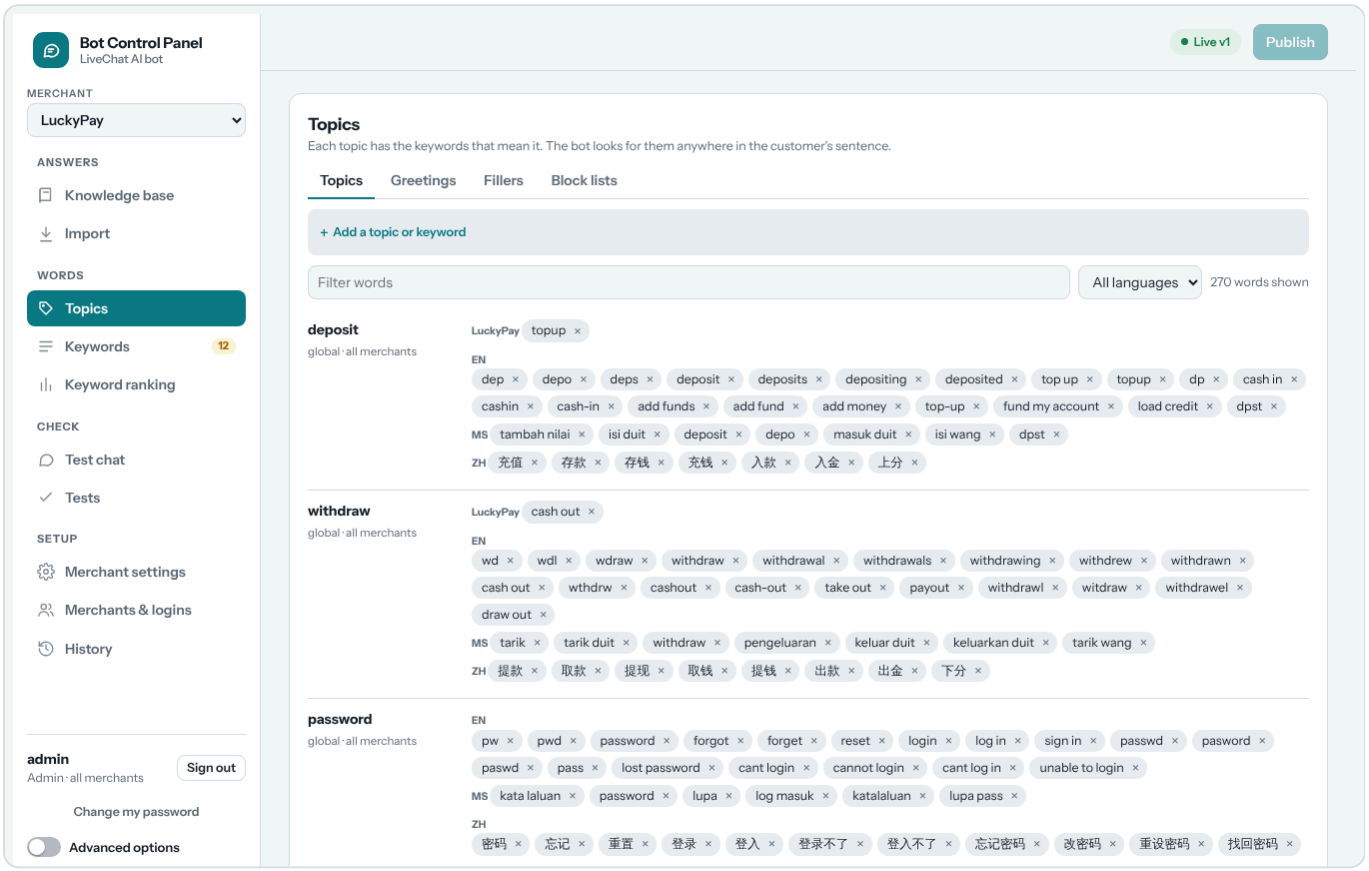



**Knowledge base**: a table of entries with topic, language dots (EN, MS, ZH), the English answer and status. Click a row to edit.

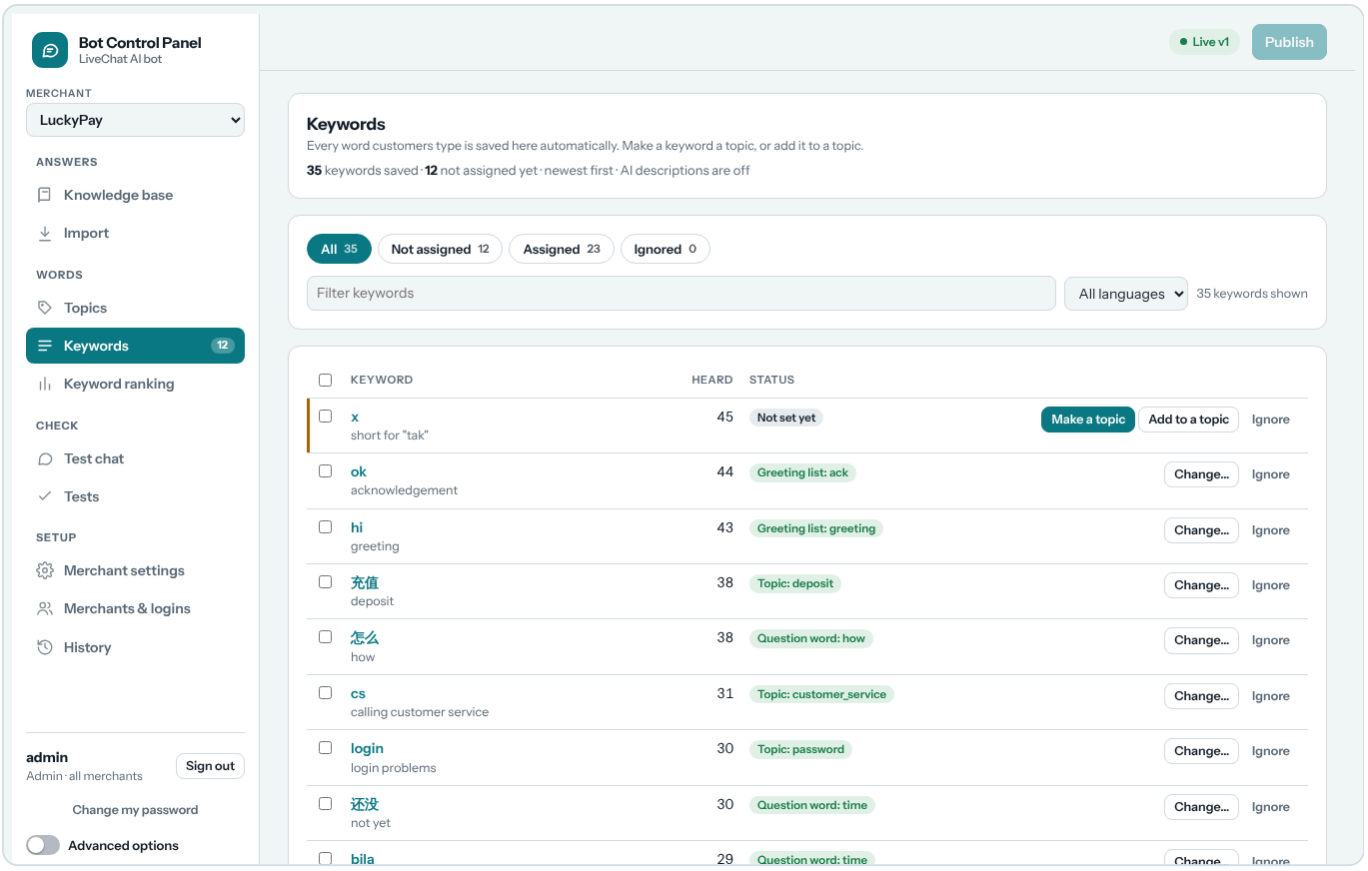
**Import**: four numbered steps (merchant, add the file, check, import).



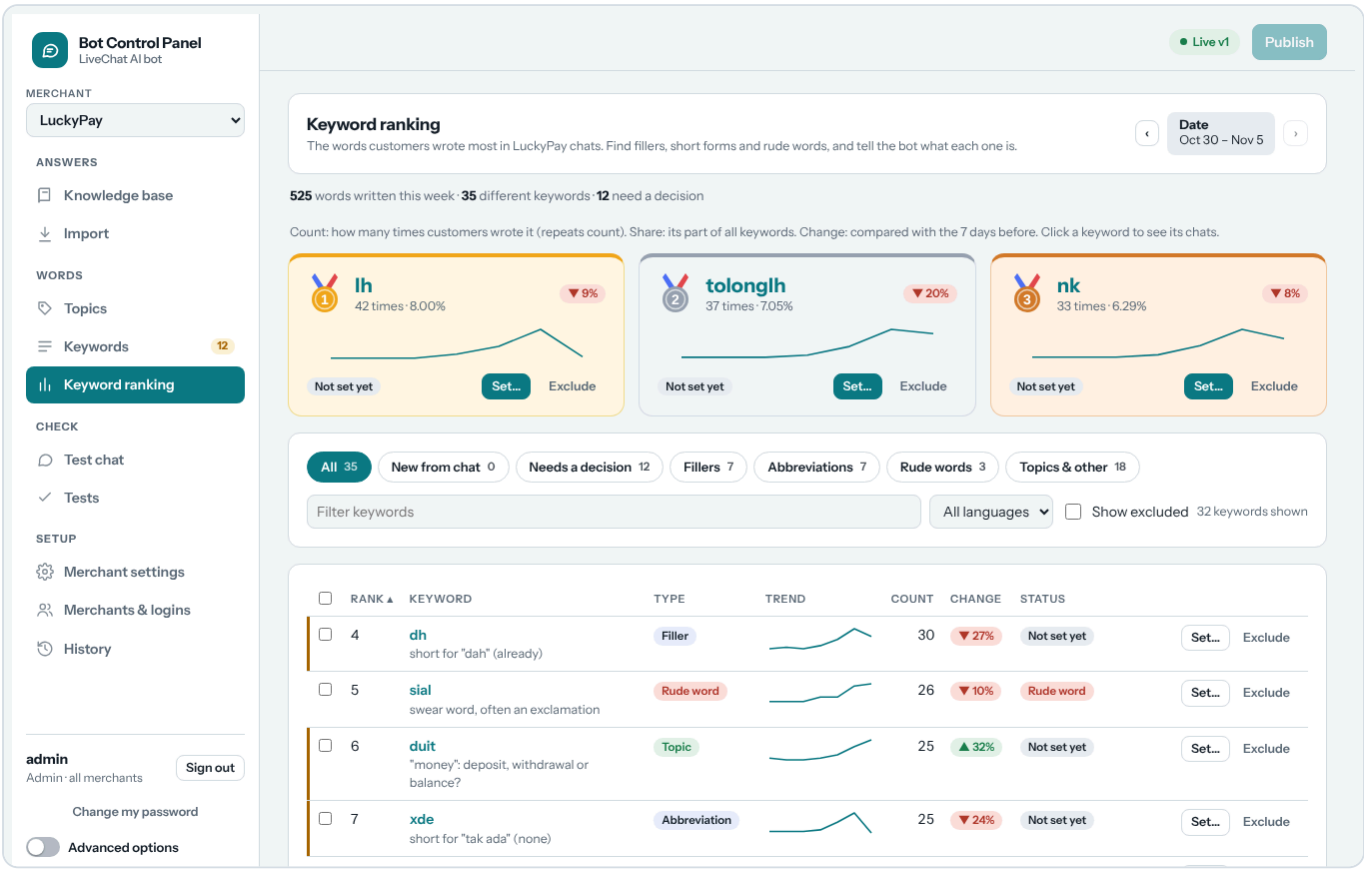

**Topics**: tabs for Topics, Question words (with Advanced on), Greetings, Fillers and Block lists. The add forms fold away until needed.



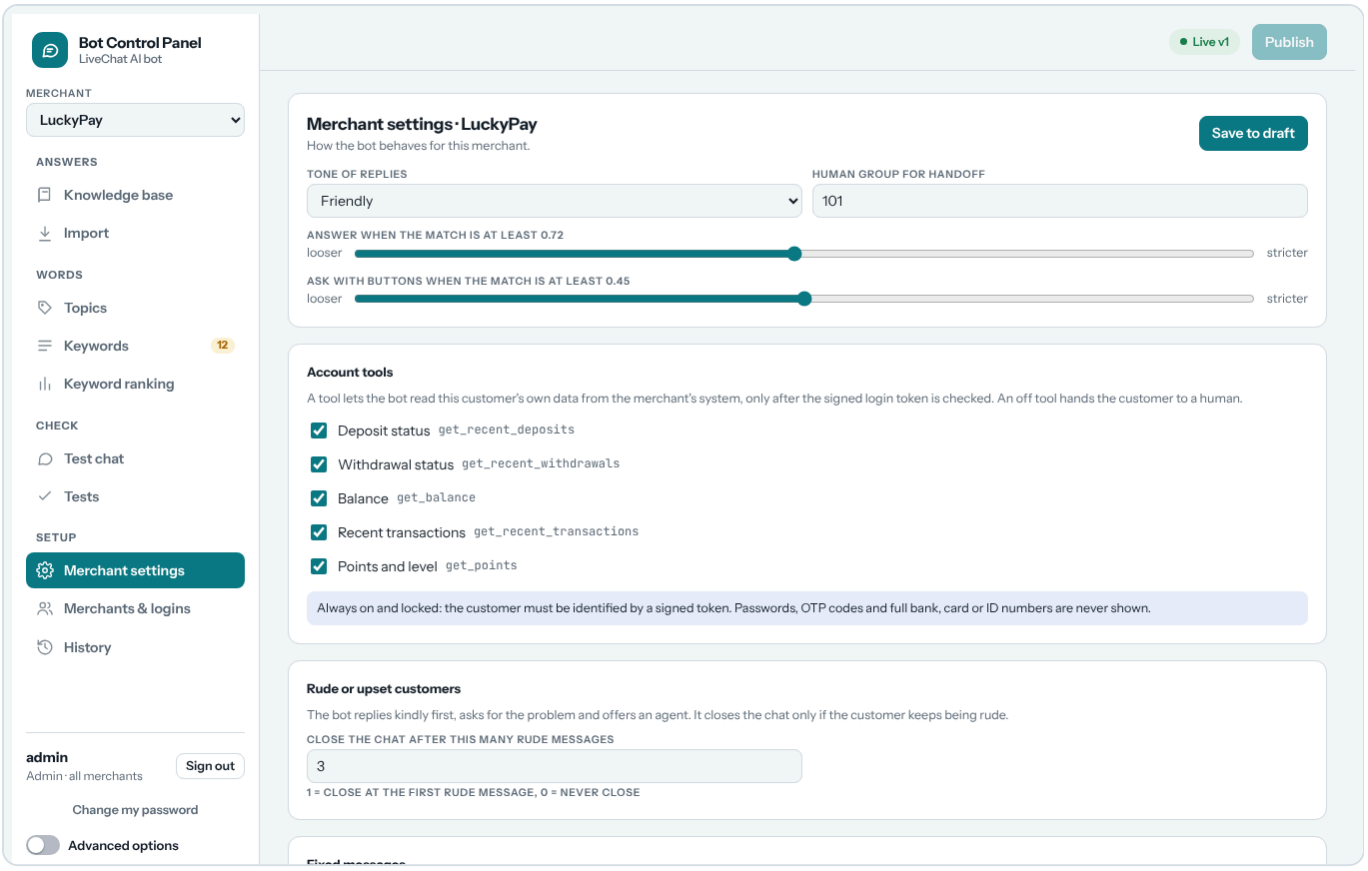

**Keywords**: every word typed, newest first, with tabs *All, Not assigned, Assigned, Ignored*.



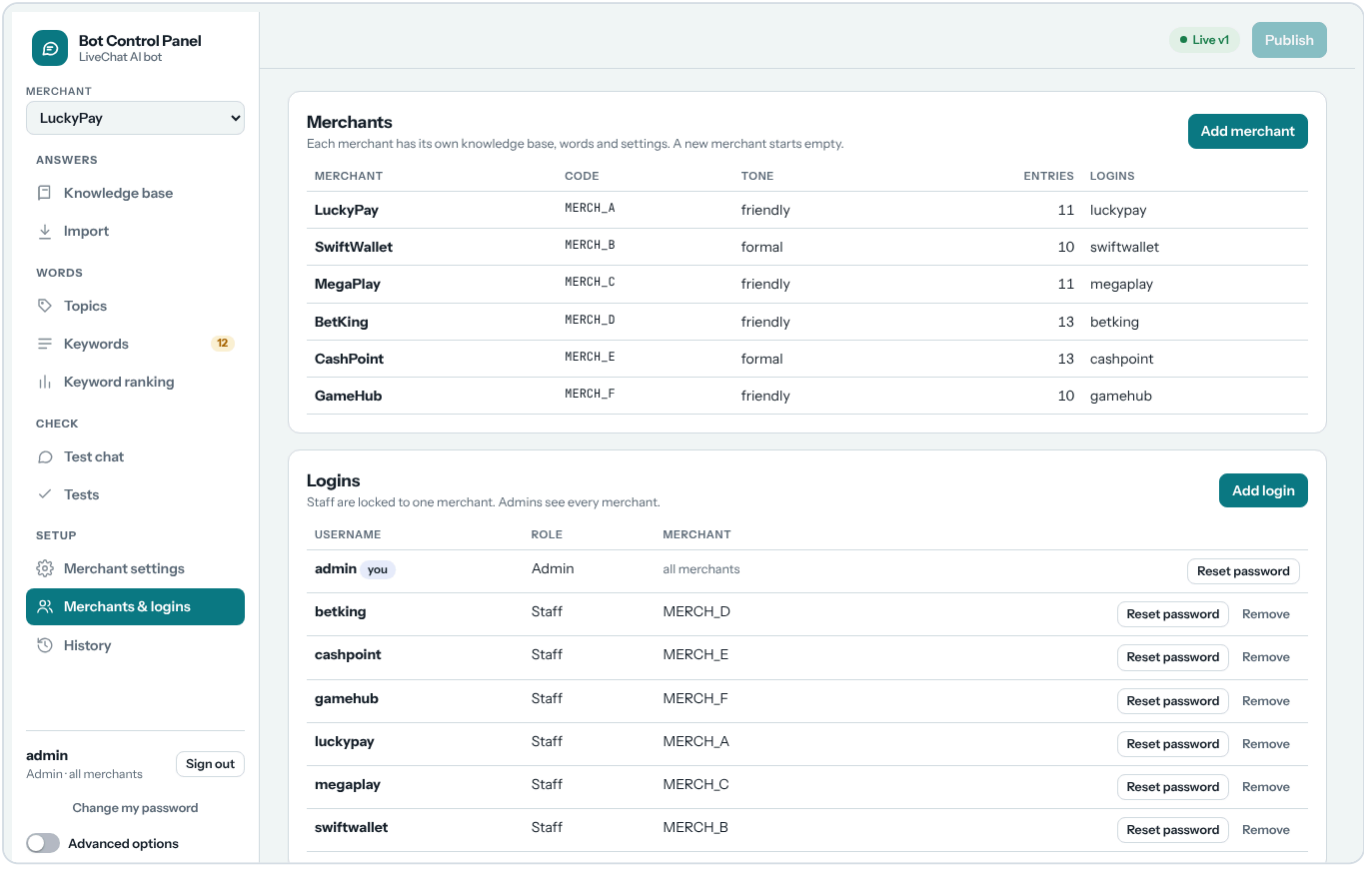

**Keyword ranking**: the top 3 as gold, silver and bronze cards, then a table. The week can be changed with the arrows.



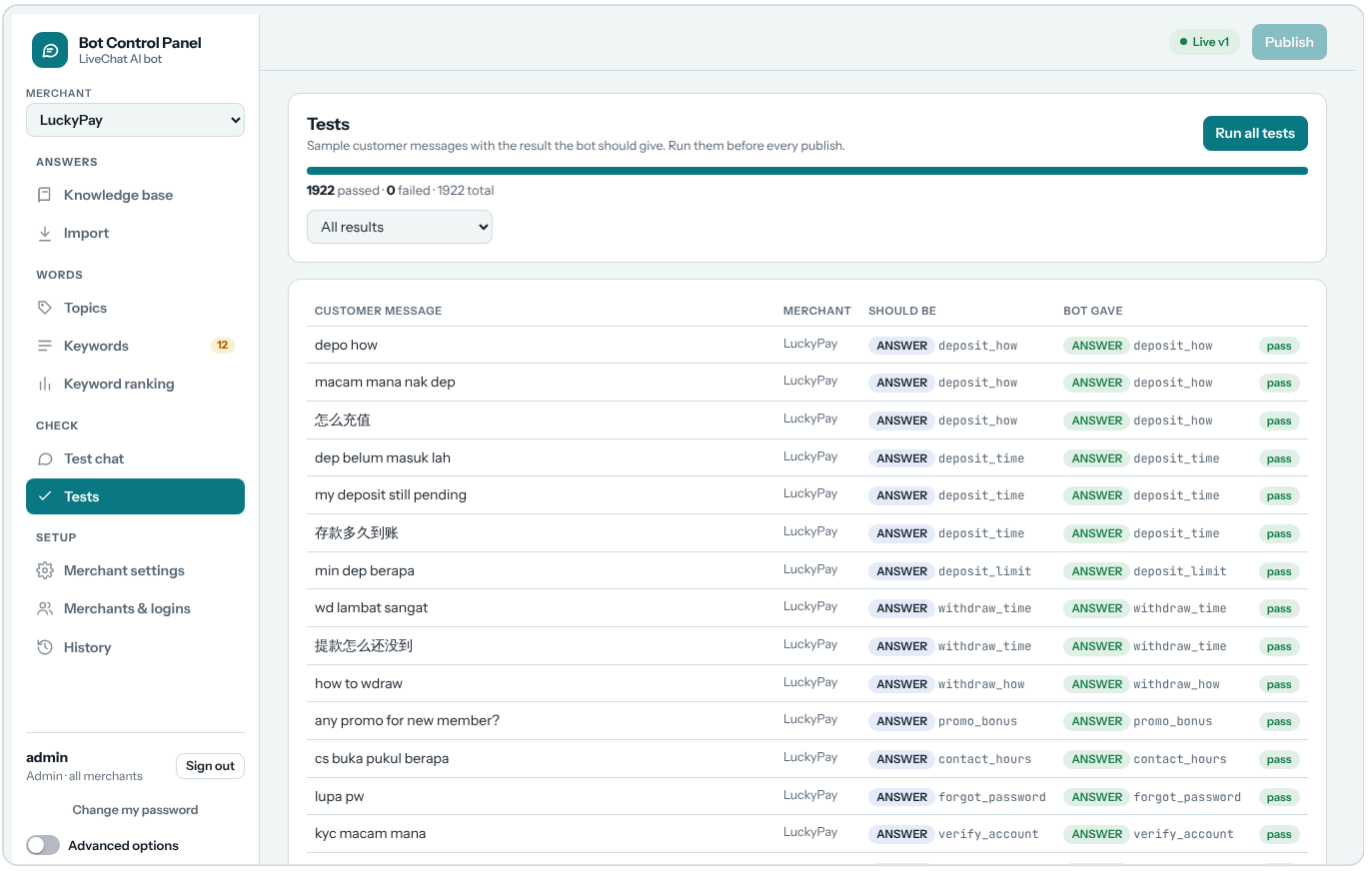

## Merchant settings



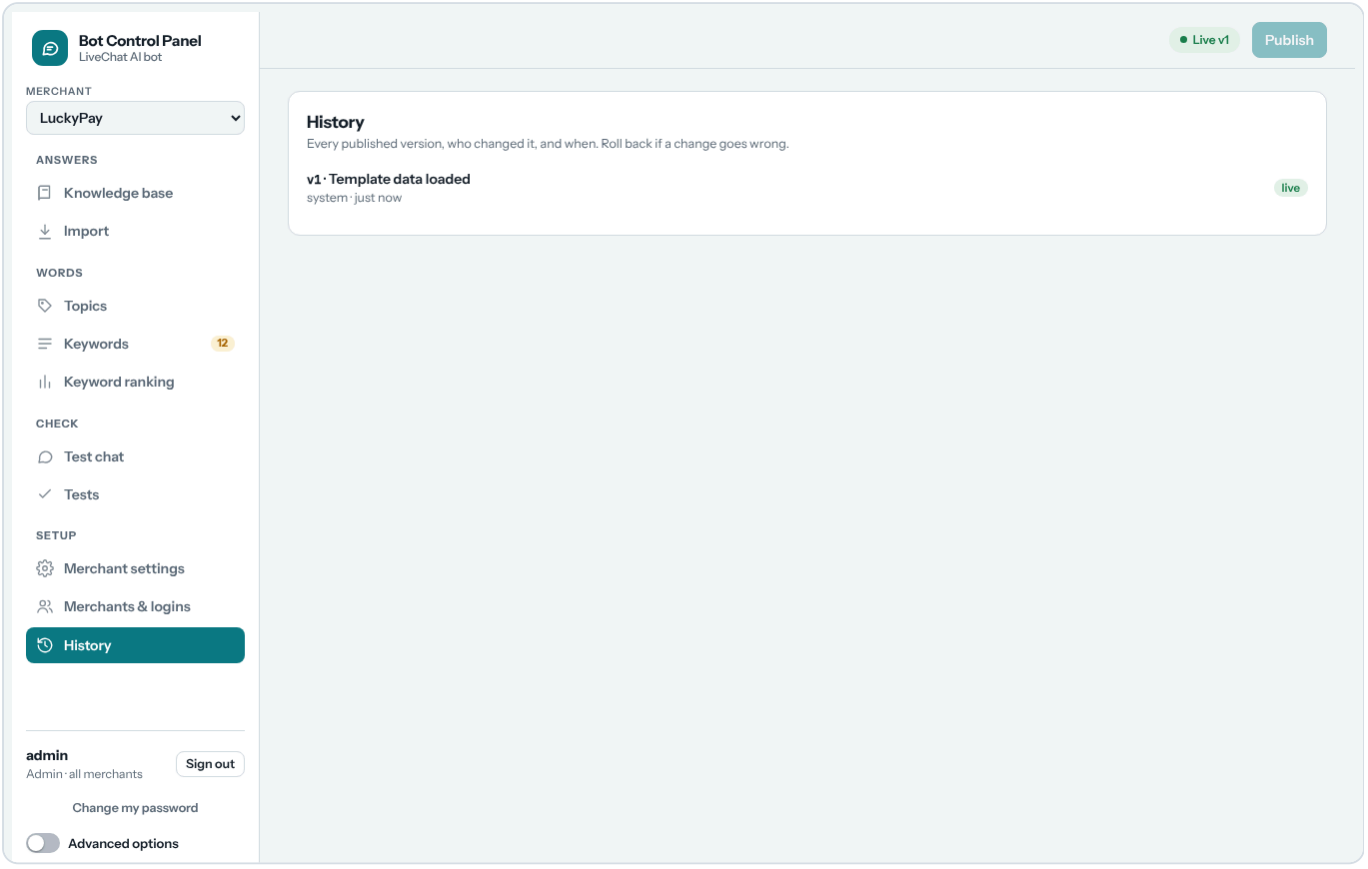

## **Merchants and logins** (admin only)




## Tests

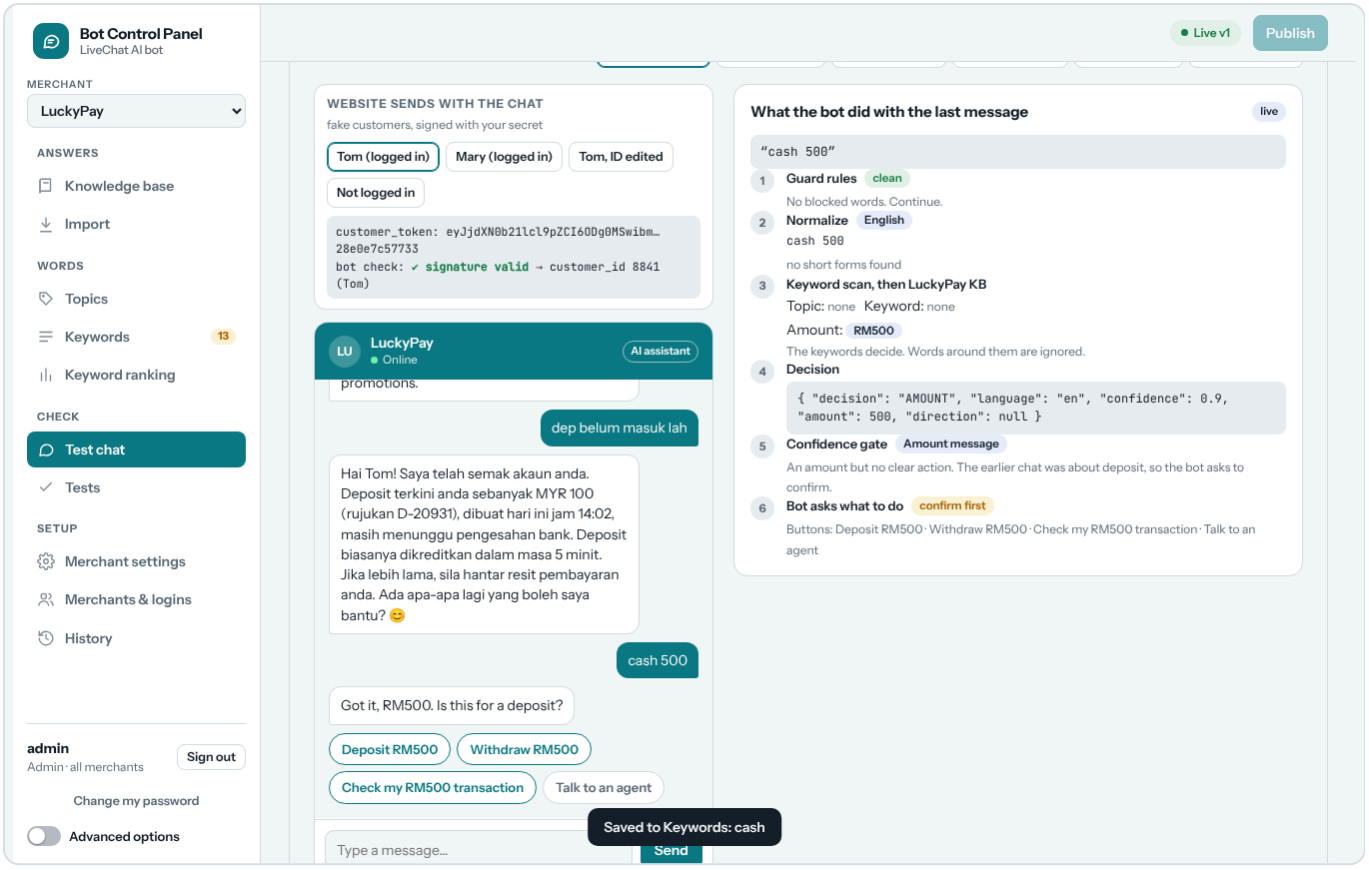

Bot Control Panel
LiveChat AI bot
MERCHANT
LuckyPay
ANSWERS
Knowledge base
Import
WORDS
Topics
Keywords
12
Keyword ranking
CHECK
Test chat
Tests
SETUP
Merchant settings
Merchants & logins
History
admin
Admin · all merchants
Sign out
Change my password
Advanced options
Live v1
Publish
Tests
Sample customer messages with the result the bot should give. Run them before every publish.
Run all tests
1922 passed · 0 failed · 1922 total
All results
CUSTOMER MESSAGE
MERCHANT
SHOULD BE
BOT GAVE
depo how
LuckyPay
ANSWER deposit_how
ANSWER deposit_how
pass
macam mana nak dep
LuckyPay
ANSWER deposit_how
ANSWER deposit_how
pass
怎么充值
LuckyPay
ANSWER deposit_how
ANSWER deposit_how
pass
dep belum masuk lah
LuckyPay
ANSWER deposit_time
ANSWER deposit_time
pass
my deposit still pending
LuckyPay
ANSWER deposit_time
ANSWER deposit_time
pass
存款多久到账
LuckyPay
ANSWER deposit_time
ANSWER deposit_time
pass
min dep berapa
LuckyPay
ANSWER deposit_limit
ANSWER deposit_limit
pass
wd lambat sangat
LuckyPay
ANSWER withdraw_time
ANSWER withdraw_time
pass
提款怎么还没到
LuckyPay
ANSWER withdraw_time
ANSWER withdraw_time
pass
how to wdraw
LuckyPay
ANSWER withdraw_how
ANSWER withdraw_how
pass
any promo for new member?
LuckyPay
ANSWER promo_bonus
ANSWER promo_bonus
pass
cs buka pukul berapa
LuckyPay
ANSWER contact_hours
ANSWER contact_hours
pass
lupa pw
LuckyPay
ANSWER forgot_password
ANSWER forgot_password
pass
kyc macam mana
LuckyPay
ANSWER verify_account
ANSWER verify_account
pass

## History

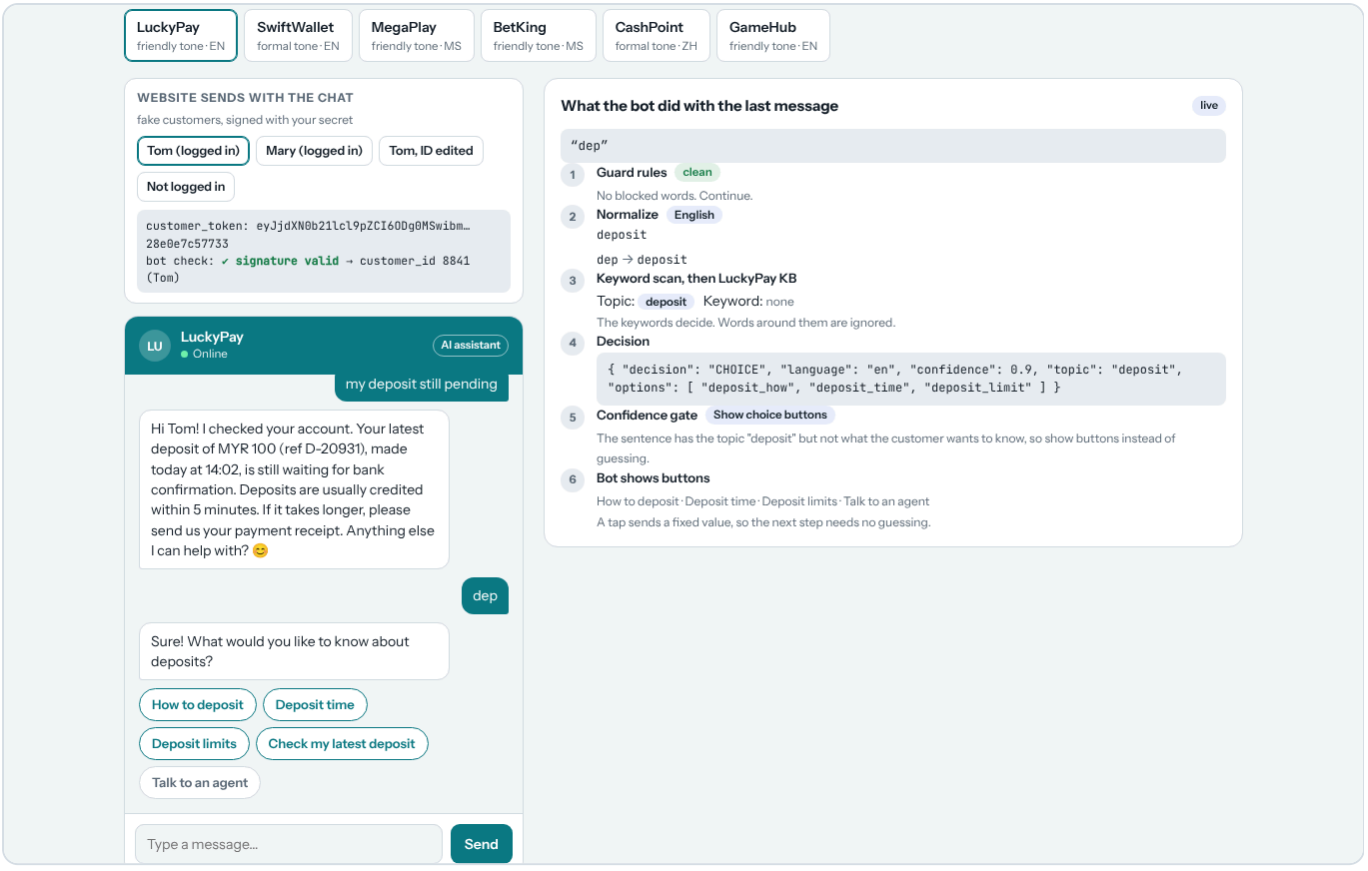

Bot Control Panel
LiveChat AI bot
MERCHANT
LuckyPay
ANSWERS
Knowledge base
Import
WORDS
Topics
Keywords
12
Keyword ranking
CHECK
Test chat
Tests
SETUP
Merchant settings
Merchants & logins
History
admin
Admin · all merchants
Sign out
Change my password
Advanced options
Live v1
Publish
History
Every published version, who changed it, and when. Roll back if a change goes wrong.
v1 · Template data loaded
system · just now
live

## Test chat

The Test chat tab shows the real chat page inside the panel, running on the draft. It has the same layout as the stand-alone chat page.



### The chat page ( `/chat` )




- **Top row**: one button per merchant. Each merchant has its own accent colour.
- **Left column**: *Website sends with the chat* (choose a fake customer: Tom, Mary, "Tom, ID edited", or not logged in, and see whether the signature is valid), the **chat window** (greeting, messages, buttons), and *Try these messages* (ready-made examples).
- **Right column**: **What the bot did with the last message**, step by step: guard rules, normalising, keywords found, the decision as data, the reason, and the reply. If Claude or ChatGPT was used, that step is shown too.

---

## 11. API addresses

All calls need a valid login cookie unless marked open. Data is sent as JSON.

| Address | What it does |
|---|---|
| `GET /health` (open) | Is the server and database up |
| `POST /api/auth/login` (open) | Sign in |
| `POST /api/auth/logout`, `GET /api/auth/me`, `POST /api/auth/password` | Sign out, who am I, change own password |
| `GET /api/merchants` | Merchants the person may use |
| `GET /api/dev/customers` | Fake customers with signed tokens (test only) |

| Address | What it does |
| --- | --- |
| `POST /api/chat/start`, `/api/chat/message`, `/api/chat/pick` | Start a chat, send a message, tap a button |
| `POST /api/analyze` | Shows what the engine decides for a message |
| `GET /api/keywords/summary` | How many words and unclear messages were saved |
| `POST /api/reload` | Reload the knowledge (admin) |
| `GET /api/admin/state` | Everything the panel needs |
| `POST /api/admin/tests` | Run the sample messages on a draft |
| `POST /api/admin/publish`, `/api/admin/rollback` | Publish (a new version for each merchant that changed), roll back one merchant or the shared lists |
| `POST /api/admin/draft` | Build a temporary bot from a draft (Test chat) |
| `GET /api/admin/keywords`, `/keywords/samples`, `/ranking` | Keyword list, example messages, weekly ranking |
| `POST /api/admin/keywords/ignore`, `/keywords/describe` | Ignore a word, describe words with AI |
| `GET /api/admin/people` (admin) | Merchants and logins |
| `POST /api/admin/merchants`, `/users`, `/users/password`, `/users/delete` (admin) | Add a merchant, add a login, reset a password, remove a login |

## 12. Folders and files

```
aibot/
  web/                the front end (React + TypeScript), built by Vite into web/dist
    index.html          the one page that loads the app
    src/main.tsx        the routes: /login, /chat, /admin/...
    src/pages/          Login.tsx, Chat.tsx, Brain.tsx (what the bot did panel)
    src/admin/          Admin.tsx (menu, top bar, publish), store.tsx (draft and loaded data),
                        model.ts (word rules, import checker), screens/ (one file per screen)
    src/lib/api.ts      calls to the bot server
    src/styles/         admin.css, chat.css, login.css
  vite.config.ts        build settings (dev server on port 5173, passes /api to the bot)
  src/
    server/             app.ts (all routes and the sign-in check), index.ts (starts it)
    engine/             engine.ts (the decision rules), types, seed adapter
    bot/                responder.ts (reply builder), texts.ts (3 languages x 2 tones),
                        token.ts (customer token), merchantApi.ts (fake account data), store.ts
    ai/                 claude.ts, openai.ts, validator.ts (fact checker), index.ts
    admin/              routes.ts, snapshot.ts (read and write the whole knowledge),
                        scope.ts (what staff may see and change), people.ts
    auth/               passwords.ts, session.ts, users.ts, addUser.ts
    db/                 pool, migrate, seed, loadKnowledge, tenants.ts (finds each merchant's
database)
    config.ts           reads .env
  db/migrations/        001 to 008 for the main database (008 moves old data into merchant
databases)
  db/merchant/          the tables every merchant database gets
  db/setup.sql          creates the databases and the bot user (run once as the MySQL admin)
  data/                 fake merchants, entries, accounts and sample messages (JSON)
  scripts/              evalReport.ts (lists wrong sample messages)
  test/                 automatic tests
  docs/                 this document
  design/               the original mockups
```

## 13. Settings (.env)

| Setting | Meaning |
|---|---|
| `DATABASE_URL` | Where the database is, for example `mysql://bot:bot@localhost:3306/livechat_bot` |
| `PORT` | Port of the web server (3000) |
| `TEST_DATABASE_URL` | A separate database for tests. It is wiped on every test run. |
| `SESSION_SECRET` | Signs the login cookie. Use a long random value. |
| `CUSTOMER_TOKEN_SECRET` | Shared with the merchant website; signs the customer token. |
| `DEV_TOOLS` | `on` shows the fake-customer buttons; set `off` for real use |
| `ANTHROPIC_API_KEY`, `CLAUDE_MODEL` | Claude (key, and model, default `claude-haiku-5-5`) |
| `OPENAI_API_KEY`, `OPENAI_MODEL` | ChatGPT (key, and the model name; empty keeps ChatGPT off) |
| `AI_MAX_CALLS_PER_HOUR` | Cost limit for AI calls (600) |

| Setting | Meaning |
|---|---|
| `LIVECHAT_*` | For the future LiveChat connection (not used yet) |

## 14. How to run it

1. Install **Node.js 20 or newer** and **MySQL 8** (or XAMPP).
2. Create the databases and the `bot` user once, as the MySQL admin: `mysql -u root -p < db/setup.sql` (see section 18 for Windows and PowerShell). Run it again after an update that adds permissions: it is safe to repeat.
3. Copy `.env.example` to `.env`.
4. `npm install`
5. `npm run db:migrate` (creates the tables) and `npm run db:seed` (loads the 6 fake merchants, 68 entries and demo logins).
6. `npm test` runs the automatic tests.
7. `npm run dev` builds the front end and starts the bot. Open **http://localhost:3000** and sign in.

Demo logins: `admin` / `admin123` (everything), and `luckypay`, `swiftwallet`, `megaplay`, `betking`, `cashpoint`, `gamehub` / `demo123` (own merchant only). Change these before real use.

Useful commands: `npm run eval:report` (lists sample messages the bot gets wrong), `npm run user:add -- <name> <password> staff <MERCHANT_CODE>` (add a login), `npm run db:reset` (wipes and reloads the fake data).

## 15. Testing and quality

- **2,017 automatic tests**, including **1,922 sample customer messages** in English, Malay and Chinese (slang, short forms and typos) across six merchants, each with the expected result.
- Tests also cover the database, the control panel API, login and the merchant lock (with forged requests), merchants and logins, the AI layer (on a fake network), and the reply builder.
- `npm run eval:report` runs all sample messages and prints every one that gives a different result than expected.

**How accurate is the bot?** The sample messages were used to tune the bot, so passing them proves little by itself. To get an honest number, two fresh batches of messages written *after* tuning were run once, before any fix:

| Fresh batch | First-try result |
|---|---|
| 49 realistic, messy messages | **82%** |
| 35 new messages, after the first round of fixes | **91%** |

Every miss was then fixed and kept as a test. A fair expectation for messages the bot has never seen is **about 85 to 90%**. Real customer messages from the live chat are the best way to raise it further.

# 16. What is not finished

| Item | State |
|---|---|
| **LiveChat connection** | Not built. Today the bot runs on a test page only. It needs the LiveChat developer details and a public web address. |
| **AI with real keys** | Built and tested only on a fake network. Real wording quality and speed are untested. |
| **Merchant API** | Account data is fake JSON. Each merchant's real API still has to be connected. |
| **Demo data** | The merchants, entries, accounts and the keyword history in the ranking are fake sample data. |
| **Chat memory** | Chat state is kept in the server's memory, so a restart ends running chats. The chat log is saved in the database. |
| **HTTPS** | Not built in. For real use, run it behind HTTPS (the cookie must then be marked secure). |
| **Hardening** | Login has a lockout, but the chat API has no speed limit yet. No automatic backups. |
| **Two admins at once** | Each publish saves a new version of the merchants that changed, so if two admins edit the same merchant at the same time, the later publish wins. |
| **Unused tables** | `feedback`, `unmatched_messages`, and the columns `business_hours`, `valid_from` and `valid_to` exist but no screen uses them. |
| **"Suggest with AI" button** | A placeholder: it only adds simple variations of the first example question. It does not call an AI. |
| **Languages in the panel** | The control panel is in English only. |

# 17. What you asked for, and where it is

| What you asked for | State | Where |
|---|---|---|
| A bot for an existing LiveChat account serving many merchants, in Malay, Chinese and English | Built and tested on a test page | Sections 4 and 5 |
| Each merchant has its own knowledge base | Done. Each merchant has its **own database**, `livechat_bot_m_<code>` | Section 9 |
| Merchants first, everything else under the merchant | Done: the main database lists merchants, and every merchant database holds its knowledge base, keywords, chats, settings, tests and versions | Section 9 |
| Each merchant publishes and rolls back on its own | Done: each merchant has its own version numbers and History | Sections 7 and 9 |
| The bot finds the answer; Claude may help choose, ChatGPT may help phrase | Built; needs your AI keys to be tried for real | Section 6 |
| Ask again once when unsure, then hand over to a person | Done | Section 4 |

| What you asked for | State | Where |
|---|---|---|
| Reply kindly first to rude customers; never show secrets | Done | Sections 4 and 8 |
| Staff are locked to their own merchant (no "sign in as", no changing merchant) | Done, checked on the server too | Section 8 |
| A real build, not plain HTML | Done: React + TypeScript + Vite | Section 3 |
| Control panel with draft, test, publish, history and roll back | Done | Section 7 |
| Keyword store: every typed word is saved; ranking with the top 3 shown | Done | Section 10 |
| Test chat that saves keywords and shows the bot's steps | Done | Section 10 |
| "Needs review" screen removed; content full width; English interface | Done | Section 10 |
| Add a merchant and its first staff login from the panel | Done | Section 7 |
| LiveChat connection | **Not built**, as you asked to do it later | Section 16 |
| Real AI keys | **Waiting for your keys** (put them in `.env`, never in chat) | Sections 6 and 13 |
| A document of the system, with the database and layout | This document | |

## 18. Installing on Windows (XAMPP) and fixing common problems

These are the problems met while installing on a Windows computer with XAMPP, and the fix for each.

**Which MySQL?** A computer can have more than one MySQL. XAMPP's runs on the port in `C:\xampp\mysql\bin\my.ini` (here **3310**), and another MySQL can run on 3306. The port in `DATABASE_URL` (in `.env`) must be the one phpMyAdmin looks at. Set it in phpMyAdmin's `config.inc.php` with `$cfg['Servers'][$i]['port'] = '3310';`.

**Importing `db/setup.sql`.** - PowerShell does not accept `<`. Use: `C:\xampp\mysql\bin\mysql.exe -u root -p -P 3310 -h 127.0.0.1 -e "source db/setup.sql"` from inside the project folder. - If XAMPP's `mysql.exe` says `caching_sha2_password could not be loaded`, you are talking to a MySQL 8 server. Use phpMyAdmin instead: log in as root, open **SQL**, paste the file. - phpMyAdmin: log in with `bot` / `bot` for the data. The red `pma` messages are harmless (comment out `controluser` and `controlpass` to hide them).

**Forgot the MySQL root password, or the `bot` user needs a new permission.** Stop MySQL in XAMPP, then in an Administrator PowerShell start `C:\xampp\mysql\bin\mysqld.exe --defaults-file=C:\xampp\mysql\bin\my.ini --skip-grant-tables --console`. In a second window: `mysql.exe -u root -P 3310 -h 127.0.0.1`, then `FLUSH PRIVILEGES;`, `GRANT ALL PRIVILEGES ON \`livechat_bot_%\`.* TO 'bot'@'localhost';`, `FLUSH PRIVILEGES;`. Then shut down **cleanly** with `mysqladmin.exe -u root -P 3310 -h 127.0.0.1 shutdown` and start MySQL from XAMPP. Do not kill the window.

**MySQL will not start: "Aria recovery failed".** This is caused by closing MySQL forcibly. Your data is in InnoDB and is not lost. Stop all `mysqld.exe` in Task Manager, copy `C:\xampp\mysql\data` somewhere as a backup, delete `aria_log.*` and `aria_log_control` in that folder, and start MySQL again. If it still fails, run `aria_chk.exe -r C:\xampp\mysql\data\mysql\*.MAI`.

**"The database user may not create the database livechat_bot_m_...".** The `bot` user lacks the permission above. Grant it as root, then run `npm run db:migrate` again. The page shows "Internal Server Error" until the migrate has finished.

**Build error `EPERM` or "Access is denied" in `web\dist`.** The folder was probably created by another Windows user (for example in an Administrator window). Delete it once from an Administrator PowerShell (`Remove-Item -Recurse -Force C:\xampp\htdocs\aibot-app\web\dist`), then run `npm run dev` from a normal PowerShell, inside the project folder. The build no longer stops if an old file is locked.

**`npm run dev` says package.json not found.** You are in the wrong folder. Run `cd C:\xampp\htdocs\aibot-app` first.